

Development Plan

Natural Control

Group 7
Aaron Loh

Three team member names pending confirmation

September 23, 2026

Review draft – September 23, 2026. Prepared from Aaron Loh’s planning interview. Team-wide ratification, remaining names, approvals, and reflections are pending. Proposed targets are not achieved results. This document is not ready for submission.

Table 1 summarizes revision history.

Table 1: Revision history.

Date	Contributor	Change
2026-09-23	Aaron Loh / AI assistance	Initial review draft based on Aaron’s planning interview; team ratification pending.

This plan describes how a four-person team proposes to build and evaluate a general-purpose, voice-supervised agent interface for blind users. It covers team operations, technical choices, the proof of concept (POC), and evaluation. It does not promise a universal replacement for screen readers. Initial use is on Mac, in English, alongside VoiceOver and keyboard access. The companion Problem Statement and Goals defines the user outcomes.

1 Confidential Information?

No external confidentiality agreement is known from Aaron’s planning interview. **Pending team input:** All members must confirm any employer, sponsor, or industry restrictions. Participant identities, recordings, account credentials, and personal task data are private even without an industry agreement. Public artifacts will use test accounts, synthetic data, and consented, de-identified findings. Raw recruitment and meeting materials remain outside the public repository. No raw microphone audio will be retained by default; provider retention policies require separate review and disclosure.

2 Intellectual Property (IP) to Protect

No external intellectual-property agreement is currently known. The intended contribution is the accessible interaction and supervision layer, its adapters, and evaluation, building on existing models and tools. The team must confirm ownership and any required institutional agreements before release. Upstream code, model weights, services, and assets retain their own terms; licensing our code does not relicense dependencies.

3 Copyright License

MIT is the proposed license for original project contributions. Its permissive terms require retaining the copyright and permission notice [1]. The repository’s [LICENSE](#) currently contains the inherited template notice. The local license proposal preserves that notice; team ratification and confirmation of attribution remain pending before publication. Third-party notices will be recorded when actual dependencies are selected, rather than assuming all candidate tools share MIT.

4 Team Meeting Plan

The proposal is one weekly team sync, supported by asynchronous work. **Pending team input:** Recurring day, time, duration, and virtual/in-person arrangement. Each meeting has a rotating chair and note taker, an issue-based agenda, attendance, decisions, owners, and next commitments recorded in GitHub. Members review materials beforehand. A member unable to attend can provide an advance written update and input on decisions; emergencies are communicated when practical. A supervisor/teaching assistant (TA) check-in approximately monthly is proposed, in addition to required course reviews; the TA’s availability and actual schedule need confirmation.

5 Team Communication Plan

Discord is proposed for coordination, with a response to actionable messages within one working day. Decisions, acceptance criteria, blockers, and assignments move to GitHub issues so they remain discoverable. Sensitive participant details and secrets do not belong in either public issues or the project board. Urgent blockers are explicitly flagged; silence is not agreement or approval.

6 Team Member Roles

The team has four members; only Aaron Loh’s name is confirmed in this draft. Primary ownership provides accountability without preventing another member from contributing. All four members implement, test, review, and document work. Owners and cross-review assignments are revisited at milestones.

Table 2 summarizes proposed workstreams.

Table 2: Proposed workstreams.

Proposed workstream	Responsibility and assignment
Voice and accessible client	Conversation controls, keyboard access, VoiceOver compatibility. Owner and cross-reviewer pending.
Agent and tool integration	Hermes adapter, account/browser tools, local execution control. Owner and cross-reviewer pending.
Deployment and reliability	Configuration, credentials, CI, reconnection, observability. Owner and cross-reviewer pending.
Evaluation and quality	Recruitment coordination, task protocol, outcome measurement, accessibility checks. Owner and cross-reviewer pending.

Meeting chair, note taker, release coordinator, and TA liaison are rotating process roles. A contributor reviews outside their primary workstream to reduce dependence on one person. **Pending team input:** Three names, named initial owners, cross-reviewers, and liaison.

7 Workflow Plan

Work is tracked in [the public repository](#). Each meaningful change starts with an issue describing the problem, acceptance criteria, owner, and relevant risk. Labels distinguish features, defects, accessibility, research, documentation, and blockers. Short-lived topic branches and small pull requests keep changes reviewable. One non-author reviewer must approve each pull request; relevant continuous integration (CI) checks must pass before merging.

The repository already contains a LaTeX/PDF and Pages workflow. Python and TypeScript checks will be added when the corresponding implementation exists: format/lint, type checks, unit/contract tests, and applicable integration tests. Passing automation does not establish accessibility. Manual VoiceOver/keyboard checks and participant evidence remain necessary. Secrets stay out of source control and logs. A change is done when its acceptance criteria, checks, documentation, and review are complete, with known limitations recorded.

8 Project Decomposition and Scheduling

A GitHub Project is proposed with Backlog, Ready, In progress, Review, and Done columns. Each member commits to concrete weekly outputs rather than fixed hours or commit quotas. Dependencies and risks are recorded on issues. The team aims for one primary active item per member and resolves blocked work at the weekly sync. **Pending team input:** Create and link the actual project board; agree initial assignments.

Drafts are ready 48 hours before deadlines. For the first milestone, the target is September 26 at 11:59 p.m. Toronto time, leaving time for human review and ratification before September 28 at 11:59 p.m. The TA’s September 23 guidance was that assessment uses the repository’s latest push at the deadline, without an Avenue upload; the knowledge base links the meeting source. Later official announcements take precedence.

Table 3 summarizes milestone overview.

Table 3: Milestone overview.

Period/date	Planned output or course milestone
September 28, 2026	Problem Statement and Goals; Development Plan including POC and charter.
October 1	First milestone peer-review issues; substantive findings, not manufactured issue counts.
October 19	Software Requirements Specification (SRS) and Hazard Analysis; discovery informs scope and safety requirements.
October–November	Working voice/agent supervision loop; instrumented POC trials; repeated discovery if recruitment succeeds.
November 2	Verification and Validation (V&V) Plan.
November 16	Design revision minus one and proposed fall Interview Report, subject to instructor approval.
November 23–December 3	Course POC demonstration window; assigned slot pending.
January 18, 2027	Design documentation; integration and accessibility refinement continue.
March 8	V&V report and proposed winter Usability Report, subject to approval.
March 31	Final documentation.
April 6	EXPO; event details to confirm.

Dates reflect the checked course calendar snapshot [8]; this is a milestone overview, not a replacement for the complete course calendar.

Resources and budget. The proposed total budget has a Canadian-dollar ceiling of \$500, not authorization to spend. A provisional allocation is \$240 participant appreciation, \$120 model/voice application programming interfaces (APIs), \$60 hosting/telephony, and \$80 reserve. Appreciation assumes at most twelve paid sessions at \$20, subject to recruitment and institutional guidance. Team Macs are already available. Test accounts and synthetic data reduce exposure of personal information. Usage metering and monthly reconciliation will track spend; paid expansion requires a revised allocation within the ceiling or an explicitly approved budget change.

9 Proof of Concept Demonstration Plan

The POC tests a narrow integration risk before broader prototype development.

9.1 Risk and experiment

The central risk is whether a blind user can understand and control a running general-purpose agent through voice without visually monitoring it. Individual speech and agent components do not establish that their integration works. The POC will test the supervision interface across three task classes, rather than implement a set of website-specific scripts:

1. **Email/calendar tools:** use synthetic correspondence to prepare a reply or calendar entry; clarify ambiguous intent and obtain approval before sending or creating the event.
2. **An unfamiliar website:** locate and compare information through general browser tools, report a meaningful choice, and incorporate a correction. The site and task are selected before the trial, without coding a dedicated workflow.
3. **A local Mac task:** locate and open a test file or interact with a supported application; prepare a derived file, confirming consequential changes such as overwrite. Native control support is an experimental dependency.

Each class exercises clarification, consequential-action approval, progress, correction, explicit execution stop, and reconnection. The demonstration uses controlled accounts and data; no participant’s real email is required.

9.2 Control semantics

Ordinary speech interruption stops spoken output while the underlying task continues. An explicit “stop working” requests cancellation of execution. The interface first acknowledges receipt, then separately confirms whether execution has stopped. Acknowledgment is not proof of cancellation. Once stopped is confirmed, no further action may be dispatched for that task. An action already completed cannot be undone by saying stop; in-flight actions with an unknown outcome must be reported and reconciled before retrying.

Disconnection pauses new actions unless the user previously authorized background continuation for that task. Reconnection presents current state and pending decisions; it must not duplicate a send or other consequential action. Navigation and preparation can proceed autonomously, but sending, purchasing, submitting, and consequential account/file changes require specific approval. If the action or its relevant details change, the previous approval does not apply.

The POC will inject a misunderstanding, interrupt speech, request stop during a tool operation, and disconnect while work is active. Logs correlate task, action, approval, cancellation, and connection events without retaining sensitive payloads. The team must verify cancellation through the adapter and downstream worker; a browser or native action may have different cancellation behavior.

9.3 Measures and participant evaluation

Targets below are provisional goals, not results:

- At least 80% correct independent completion on a declared representative task set, following assisted setup. Correct means the specified outcome was achieved and verified without researcher task assistance. Report numerator, denominator, task categories, failures, and any assistance separately.
- Stop acknowledgment within two seconds, measured from the end of a recognized explicit stop utterance to audible acknowledgment. Record recognition failures separately and measure execution-stop confirmation latency independently; no two-second execution-cancellation guarantee is claimed.
- Approval before every tested consequential action, checked against action and approval traces, including attempts that should be blocked.

- Assess whether users can identify completion or a blocker and correct a seeded misunderstanding. Record errors, clarification turns, time, perceived control, and useful versus excessive spoken feedback.

The team seeks four to six blind participants with screen-reader experience for repeated discovery and evaluation. Recruitment is unconfirmed. The protocol, consent and institutional research requirements must be resolved before sessions. A personal route to an accessibility organization is a possible introduction, not a partnership or recruitment commitment.

Before the main evaluation, freeze task definitions, outcome checks, allowed assistance, and exclusions. Report all attempted tasks and explain invalidated trials. Compare participants’ usual screen-reader workflow with the assistant where feasible, varying order to reduce learning effects. A small convenience sample supports formative findings, not population-wide efficacy claims. Technical POC results and participant results will be reported separately.

9.4 Failure consequences

Table 4 summarizes pOC uncertainties and failure consequences.

Table 4: POC uncertainties and failure consequences.

Uncertainty	Evidence needed and consequence if unresolved
Harness supervision	Verify streamed status, approvals, steering, cancellation, and reconnect on a pinned Hermes version. If unavailable, revisit the adapter/harness before promising supervision.
Tool cancellation	Test slow and in-flight account/browser operations. If outcome is unknown, expose that state and block retries; do not claim confirmed stop.
Native Mac control	Prove permissions, actionable targets, and result verification. If unsuccessful, defer native control and retain general browser/account tasks.
Voice and VoiceOver	Measure latency, intelligibility, and competing audio with keyboard alternatives. If remote calls fail, retain local voice.
Remote continuity	Test network loss, pending approvals, and duplicate prevention. Defer telephone interaction if it cannot preserve control.
Recruitment	Confirm access early. Without participants, report technical feasibility only and disclose the missing usability evidence.

10 Expected Technology

The proposed architecture is:

Accessible TypeScript/React web client → LiveKit/Python integration → Hermes → account tools, Browser Use, and local Mac tools.

The team will implement the accessible task/decision presentation and supervision adapter, reusing existing speech, agent, and tool implementations. One supported default configuration will be

documented and version-pinned, with extension points and user-provided credentials. Provider choices and exact versions remain pending POC evidence. No model training is planned.

10.1 Agent and action tools

Hermes is the primary harness candidate. Its documented programmatic interface includes streaming events, steering, interrupt requests, and server requests for clarification/approval [6]. These are promising integration primitives, not proof that our end-to-end control semantics already work. The POC must verify actual behavior and failure handling on a pinned release. Codex app server is an alternative with a custom-client integration surface [7]; OpenClicky is an implementation reference. Neither establishes that arbitrary desktop plugins or account integrations transfer to our client. OpenClaw is excluded from the proposed stack.

Browser Use is the intended general browser component through Hermes. Hermes documents a Browser Use path and fallback browser mechanisms [5]. Packaged browser sessions must not be assumed to share a user’s logged-in Chrome profile. Authentication, signed-in browser attachment, permission boundaries, and compatibility need separate trials. Direct account APIs can offer clearer structured outcomes than visual clicking; browser tools remain necessary where suitable APIs are unavailable. Account connectors must be selected and verified individually, including scopes and credential handling.

Local Mac tools require an adapter with explicit permissions and observable action outcomes. Screenshot/accessibility-based drivers, including approaches used by OpenClicky, are candidates for investigation. No native driver has been selected or validated. Broad shell access is not a substitute for a verified consequential-action boundary.

10.2 Voice, client, and deployment

LiveKit with Python is proposed for audio sessions and tool integration [3, 4]. Python aligns with the anticipated agent-side integration. A TypeScript-only voice backend is an alternative that reduces language count; the Python choice adds a typed cross-process boundary to test. A voice tool call does not itself guarantee cancellation of downstream work.

TypeScript/React is proposed for the web client, using semantic controls, visible text/history, focus management, and full keyboard operation. Familiar web tooling and iteration speed favor it over an initial Swift-only client. React does not guarantee accessibility; VoiceOver testing is required. The web interface and local service support assisted setup followed by independent daily use.

ElevenLabs remains a speech-output candidate through LiveKit’s documented plugin [2]. Alternatives include other supported speech providers and native system speech. Selection will compare intelligibility, interruption behavior, latency, privacy, and actual cost. Recognition and general reasoning/vision models will likewise use existing providers, with versions and usage recorded. Local models are an option to investigate, not a commitment to fully local performance or zero cost.

Jev is optional for structured evaluation or routing [10]. Its documented role does not establish suitability as the primary conversational or visual-control model. A deterministic policy is the simpler alternative for mandatory approval rules. Promotional pricing is not a basis for the project budget.

Deployment remains hybrid: a local Mac, hosted services, or their combination. Native tasks require an online local component with user-granted permissions. Hosting the voice/agent services does not remotely confer access to a Mac. The default topology, secret storage, provider retention, and network requirements will be documented after a working configuration is demonstrated. A second voice channel connecting telephone and on-Mac interaction is the leading stretch goal.

10.3 Engineering and measurement tools

Git and GitHub provide version history, issues, reviews, and project tracking. GitHub Actions supplies repeatable checks; hosted CI cannot substitute for Mac permission tests or participant evaluation. Proposed Python tools are Ruff, pytest, and pytest-cov; proposed client tools are TypeScript strict checks, ESLint, Prettier, Vitest, and Testing Library. Final configurations and versions will be pinned during setup. Coverage is diagnostic, not proof of correctness.

Contract tests will exercise approval, cancellation, reconnection, and task-state boundaries using controlled workers, followed by real-tool integration trials. Deterministic tests do not imply deterministic website workflows in the product. Monotonic timestamps will measure voice response and stop latency; task traces will measure success, errors, and cost per completed task. A simple instrumented trace is preferable initially to a complex observability service. The team must learn the harness protocol, asynchronous audio integration, and accessible focus management through small tested integration slices.

11 Use of AI and LLMs

The product uses existing models to interpret goals and operate tools. The team also proposes AI assistance for research, code, tests, and documentation, with human understanding and review. Codex has assisted this draft; other members' tools and versions remain to be reported. Generated code must pass relevant checks and review. Sources must be opened and verified, claims distinguished from assumptions, and participant quotations/results never generated as evidence. Per-document disclosures record material assistance, affected sections, tool versions where known, and actual verification [9].

12 Coding Standard

Python will follow PEP 8 conventions, type annotations at public boundaries, and a shared formatter/linter configuration. TypeScript will use strict types and shared formatting. Small modules separate voice, task control, and tool adapters; state transitions and error outcomes are explicit. Tests cover behavior and failure boundaries. Semantic HTML, accessible names, keyboard operation, and focus behavior are part of review. Credentials and participant data are excluded from code, fixtures, and ordinary logs. Exceptions to conventions are documented in the relevant pull request.

Appendix A: Generative AI Use Disclosure

Tool and use. Aaron Loh used OpenAI Codex with GPT-6 Astra, primarily at Medium reasoning and occasionally High (settings reported by Aaron). Codex helped extract questions from course requirements, challenge assumptions, research options, and develop ideas through discussion. It also assisted with drafting and revising text throughout this document, organizing references, and LaTeX/PDF formatting. Aaron made the judgment calls and reviewed the work. These are working drafts for further team revision, not evidence of team-wide agreement or completed participant research.

Verification and responsibility. The assistant checked cited sources, course requirements, document consistency, and PDF rendering. These checks do not establish that the proposed system works. Team review, ratification, and personal reflections remain pending. Before submission, the team will verify the final content and update this disclosure to reflect subsequent revisions and any additional AI use. The team remains responsible for the submitted work.

Appendix B: Reflection

These answers must come from the members' actual experience, not invented AI reflections.

1. Why is it important to create a development plan before starting? **Pending team input:** Members' reflections on this project's planning experience.
2. What are the advantages and disadvantages of continuous integration and deployment (CI/CD) in your opinion? **Pending team input:** Members' own assessment, including actual experience where available.
3. What disagreements occurred in this deliverable and how were they resolved? **Pending team input:** An honest account of team discussion; do not claim consensus or disputes that have not occurred.

Appendix C: Team Charter

Status: proposed by Aaron during planning; awaiting ratification by all four members. These are proposed working agreements, not evidence of assent.

External goals

Produce a useful open-source contribution, a credible working prototype informed by blind users, strong engineering evidence, and a strong course outcome.

Attendance

Expectations. Attend weekly syncs prepared and on time. Give advance notice of lateness, early departure, or absence when possible. A written update and decision input can support flexible attendance; absence alone is not a failure to contribute.

Acceptable excuses. Illness, emergencies, accessibility needs, and communicated academic/work conflicts merit rescheduling or reassignment. Repeated unexplained silence or uncommunicated missed commitments calls for a recovery conversation, rather than an assumption about personal motivation.

Emergency process. Notify the group when practical, identify work at risk if able, and let the group redistribute urgent tasks. Personal details need not be made public.

Accountability and teamwork

Quality. Weekly commitments specify outputs and acceptance criteria. Meeting preparation includes reading the agenda and relevant drafts. Contributions include evidence of checks, understandable documentation, and a reviewable change. Drafts are ready 48 hours before deadlines. All members must understand and be able to explain AI-assisted work they submit.

Attitude. Discuss ideas respectfully, make room for disagreement, and ask before assuming a teammate's intent. Treat participant expertise as evidence; avoid speaking for blind people without consulting them. Do not disclose personal participant information to resolve team debates.

Staying on track. Use weekly output commitments and the issue board, not fixed hours or commit quotas. Two unexplained missed commitments in one month trigger a private recovery conversation with an explicit revised plan. If the problem remains unresolved, involve the TA with factual records and an opportunity for the member to respond. Recognize useful contributions and help remove blockers.

Team building. Include short demos and informal check-ins in the weekly sync. Recognize progress; any additional social activities are optional and have not yet been agreed.

Decision making. Seek evidence and run a small experiment where useful, then vote if discussion does not resolve the question. For a tied technical choice, the assigned owner decides and records the rationale. Course-policy questions go to the TA. No individual can invent another member's consent or participant research results.

Ratification. Pending team input: All four names, review date, acceptance or amendments, and initial responsibilities. No signatures or approval dates are implied.

References

- [1] GitHub. Mit license: Choose a license. <https://choosealicense.com/licenses/mit/>, n.d. Accessed September 23, 2026.
- [2] LiveKit. Elevenlabs tts plugin guide. <https://docs.livekit.io/agents/models/tts/elevenlabs/>, 2026. Accessed September 23, 2026.
- [3] LiveKit. Voice ai quickstart. <https://docs.livekit.io/agents/start/voice-ai/>, 2026. Accessed September 23, 2026.
- [4] LiveKit. Tool definition and use. <https://docs.livekit.io/agents/logic/tools/>, 2026. Accessed September 23, 2026.
- [5] Nous Research. Hermes agent: Browser automation. <https://hermes-agent.nousresearch.com/docs/user-guide/features/browser/>, 2026. Accessed September 23, 2026.
- [6] Nous Research. Hermes agent: Programmatic integration. <https://hermes-agent.nousresearch.com/docs/developer-guide/programmatic-integration>, 2026. Accessed September 23, 2026. Documentation, not a tested integration.
- [7] OpenAI. Codex app server. <https://learn.chatgpt.com/docs/app-server>, 2026. Accessed September 23, 2026.
- [8] SFWRENG 4G06. Course calendar. <https://calendar.google.com/calendar/embed?src=rnb0qiakik2la7rpu3bn0um58%40group.calendar.google.com>, 2026. Snapshot checked September 23, 2026; subsequent announcements may change dates.
- [9] Spencer Smith. Sfwreng 4g06 course outline, 2026. August 31, 2026. User-supplied course document; local source indexed in refs/course/README.md.
- [10] Vercel. Jev: Api, pricing and playground. <https://vercel.com/ai-gateway/models/jev>, 2026. Accessed September 23, 2026. Promotional pricing is temporary.